

[INSERTAR LOGO DEL INSTITUTO]

INFORME DE EVALUACIÓN — UNIDAD 3

Desarrollo de la capa Frontend de un Gestor Documental

Escuela Básica G-733 Chorombo Bajo, comuna de María Pinto

Antecedente	Detalle
Asignatura	Desarrollo Frontend
Unidad evaluada	Unidad 3 — Componentes, interactividad y consumo de servicios
Estudiante	Luis Orellana Cabrera
Modalidad	Trabajo individual
Docente	[COMPLETAR]
Fecha de entrega	[COMPLETAR]
Repositorio	github.com/Luisorellanacabrera/gestor-documental-frontend

Cliente: Escuela Básica G-733 Chorombo Bajo — Contraparte: Patricia Gajardo Ordenes,
Directora

Índice

Índice.....	2
1. Introducción	4
1.1 Framework y tecnologías	4
1.2 Ejecución de la solución	4
2. Propuesta de componentes.....	5
2.1 Criterios de reutilización	5
3. Resolución de un problema técnico	6
3.1 El problema.....	6
3.2 Causa detectada.....	6
3.3 Alternativas consideradas.....	7
3.4 Solución implementada	7
3.5 Resultado obtenido.....	7
3.6 Aprendizaje generado.....	7
4. Accesibilidad y usabilidad	8
4.1 Criterios WCAG 2.1 aplicados.....	8
4.2 Prueba de accesibilidad realizada	8
4.3 Prueba de navegación por teclado.....	9
4.4 Revisión de usabilidad	9
5. Integración y despliegue de los componentes	9
5.1 Verificación del funcionamiento conjunto.....	10
5.2 Preflight CORS	10
5.3 Variables de configuración	10
5.4 Instalación, ejecución y despliegue	11
6. Buenas prácticas aplicadas al proyecto.....	11
7. Control de versiones	12
8. Diseño de la interfaz de usuario.....	13
8.1 Operaciones disponibles.....	13

8.2 Información del listado	13
8.3 Retroalimentación visual	13
8.4 Diseño adaptable	14
9. Estrategias de optimización	14
9.1 Búsqueda con pausa	14
9.2 Catálogo de tipos en memoria	14
9.3 Delegación de eventos	15
9.4 Un solo redibujado de la tabla	15
10. Integración del framework en la interfaz	15
11. Consumo de endpoints	16
11.1 Manejo de solicitudes, estados y errores	17
12. Estrategias de seguridad	17
12.1 Alcance actual y limitación declarada	18
13. Documentación y difusión del conocimiento	18
14. Retrospectiva y mejora continua	19
14.1 Cómo se tomó la decisión	19
14.2 Oportunidades de mejora identificadas	19
14.3 Mejora implementada dentro del alcance	20
15. Conclusiones	20
16. Declaración de uso de Inteligencia Artificial	21
16.1 Tareas en que se utilizó	21
16.2 Justificación técnica del uso	21
16.3 Alcance y responsabilidad	22
17. Referencias	22

Para actualizar el índice en Word: clic sobre la tabla y presionar F9.

1. Introducción

La Escuela Básica G-733 Chorombo Bajo, de la comuna de María Pinto, atiende a 208 estudiantes desde prekínder hasta 8.º básico. Su equipo directivo administra de forma manual la documentación institucional: memos, oficios, citaciones de apoderados, acuerdos de apoderados, documentos de reuniones comunales y permisos administrativos. Según la ficha del actor de interés, la directora planteó la necesidad de centralizar esa información en una plataforma web, señalando que la falta de recursos económicos obliga a optimizar los medios disponibles: no existe infraestructura previa y el único equipo del establecimiento es un notebook de portería con Intel Core i5, 8 GB de RAM y 128 GB de almacenamiento.

Este informe documenta la construcción de la capa Frontend del Gestor Documental: una aplicación web que permite al equipo directivo visualizar, buscar, filtrar, registrar, modificar y eliminar documentos, consumiendo la API desarrollada en la asignatura Desarrollo Backend. Ambas capas forman el mismo proyecto.

Esas restricciones del cliente se tradujeron en decisiones técnicas concretas: la aplicación no requiere instalación ni proceso de compilación, no depende de gestores de paquetes y Bootstrap se sirve desde una copia local, de modo que funcione aunque la escuela quede sin conexión a internet.

1.1 Framework y tecnologías

Tecnología	Uso en el proyecto
HTML5 semántico	Estructura de la interfaz y base de la accesibilidad
Bootstrap 5.1.1, copia local	Rejilla, tabla, ventanas modales, avisos y diseño adaptable
JavaScript ES Modules	Componentes, manejo de estado y eventos, sin compilación
Fetch API	Consumo de los endpoints del Backend

Tabla 1. Tecnologías utilizadas y su rol en el proyecto.

Se mantuvo el framework de trabajo definido en la asignatura y empleado en la Unidad 2, de modo que la entrega sea consistente con lo trabajado durante el curso.

1.2 Ejecución de la solución

Con el Backend en funcionamiento, la aplicación se levanta con cualquier servidor estático:

```
python3 -m http.server 5500
# La aplicación queda en http://127.0.0.1:5500
```

Debe abrirse mediante un servidor web y no con doble clic sobre index.html: el proyecto usa módulos de JavaScript, que el navegador bloquea por seguridad bajo el protocolo file://.

2. Propuesta de componentes

Indicador de logro: *Propone la creación de componentes.*

Antes de programar se definió la estructura de componentes de la aplicación. El criterio que se siguió fue que cada componente hiciera una sola cosa y no conociera el funcionamiento interno de los demás: la comunicación pasa siempre por `app.js`, único módulo que conoce a todos. Esto permite modificar o reemplazar un componente sin tocar el resto.

Componente	Responsabilidad	Información que recibe	Relación con otros
Navegación (<code>index.html</code>)	Encabezado con el nombre del sistema, el estado de la conexión y la acción principal	—	Su botón «Nuevo documento» pide a <code>app.js</code> abrir el formulario
<code>resumen.js</code>	Mostrar cuántos documentos hay de cada tipo	Objeto de estadísticas de la API	Al pulsar una tarjeta avisa a <code>app.js</code> , que aplica el filtro por ese tipo
<code>barraFiltros.js</code>	Recoger texto buscado, tipo y rango de fechas	Catálogo de tipos y la función a ejecutar al cambiar	Entrega los filtros a <code>app.js</code> , que los envía a la API
<code>listaDocumentos.js</code>	Dibujar la tabla y la paginación; avisar qué acción se pulsó	Documentos, paginación y funciones de Ver, Editar y Eliminar	Recibe datos de <code>app.js</code> ; sus acciones abren detalle, formulario o confirmación
<code>detalleDocumento.js</code>	Mostrar toda la información de un documento	El objeto documento	Se abre desde el listado y puede encadenar con el formulario
<code>formularioDocumento.js</code>	Capturar y validar datos, al crear y al editar	Catálogo de tipos y, al editar, el documento	Devuelve los datos a <code>app.js</code> ; muestra los errores que responda la API
<code>confirmarEliminar.js</code>	Pedir confirmación antes de borrar	El documento a eliminar	Confirma a <code>app.js</code> , que ejecuta el borrado
<code>mensajes.js</code>	Comunicar estado: cargando, sin resultados, error y avisos de éxito	Textos	Lo usan todos los flujos de <code>app.js</code>
<code>utilidades.js</code>	Funciones compartidas: escapar HTML, formatear fechas, retrasar la búsqueda	—	Lo usan casi todos los componentes

Tabla 2. Componentes propuestos, su responsabilidad y sus relaciones.

2.1 Criterios de reutilización

- **Un solo formulario para crear y editar.** `abrirParaCrear()` y `abrirParaEditar()` comparten estructura, validación y envío. Escribir dos formularios casi idénticos habría obligado a corregir cada error en dos lugares.

- **Retroalimentación centralizada.** Ningún componente escribe avisos por su cuenta: todos pasan por `mensajes.js`, de modo que se vean y se comporten igual en toda la aplicación.
- **Protección aplicada en un solo lugar.** `escaparHtml()` vive en `utilidades.js` y la usan todos los componentes que pintan datos, para que la seguridad no dependa de recordarla caso por caso.

3. Resolución de un problema técnico

***Indicador de logro:** Resuelve problemas complejos y multifacéticos en el desarrollo de componentes.*

3.1 El problema

Desde la ventana de detalle, el botón «Editar» debía cerrar esa ventana y abrir la del formulario. Al probarlo, en algunas ejecuciones el formulario no se abría: la ventana de detalle quedaba abierta y el botón parecía no responder. El comportamiento no era constante, lo que dificultaba identificar la causa.

3.2 Causa detectada

Dos ventanas modales de Bootstrap no pueden estar abiertas a la vez: cada una crea su propia capa de fondo. La primera versión del código cerraba la ventana y después registraba el escuchador del evento de cierre:

```
modal.hide();  
modalElemento.addEventListener('hidden.bs.modal', () => alEditar(id), { once: true });
```

Bootstrap demora unos 300 milisegundos en la animación de cierre, de modo que el escuchador alcanzaba a registrarse y todo parecía funcionar. Pero cuando el cierre ocurre al instante —que es exactamente lo que sucede cuando el usuario tiene activada la opción «reducir animaciones» de su sistema operativo— el evento se dispara antes de que el escuchador exista, se pierde, y el formulario no se abre nunca.

El fallo se reprodujo quitando la clase `fade` de las ventanas, que produce el mismo efecto que esa opción de accesibilidad. Adicionalmente se midió qué ocurre si simplemente se abre la segunda ventana sin esperar a que se cierre la primera:

Escenario probado	Capas de fondo en pantalla	Al cerrar todo
Cerrar y abrir sin esperar	2 capas superpuestas	Queda 1 capa pegada y el desplazamiento bloqueado: la página se ve con un velo gris y hay que recargar
Solución implementada	1 capa, la correcta	La página queda utilizable

Tabla 3. Medición del comportamiento con y sin la solución.

3.3 Alternativas consideradas

1. **Esperar con un temporizador fijo de 300 milisegundos.** Descartada: fija a mano un tiempo que depende de la animación de Bootstrap y de la velocidad del equipo. En un computador lento vuelve a fallar, y en la escuela el equipo disponible es modesto.
2. **Usar una sola ventana que cambie de contenido.** Habría evitado el problema, pero obliga a mezclar el detalle y el formulario en un mismo componente, perdiendo la separación de responsabilidades que ordena todo el proyecto.
3. **Registrar el escuchador antes de cerrar.** Alternativa elegida: no depende de tiempos y mantiene los componentes separados.

3.4 Solución implementada

```
// El escuchador se registra ANTES de llamar a hide().
modalElemento.addEventListener('hidden.bs.modal', () => alEditar(id), { once: true });
modal.hide();
```

Invertir las dos líneas garantiza que el escuchador exista sin importar cuánto demore el cierre. La opción { once: true } evita que se acumulen escuchadores al usar el botón varias veces.

3.5 Resultado obtenido

Verificado con las animaciones desactivadas, que era el escenario que fallaba:

Comprobación	Resultado
La ventana de detalle se cierra	Sí
La ventana de formulario se abre	Sí
Capas de fondo en pantalla	1, la correcta
Contenido de la ventana	«Editar documento», con los datos del documento cargados

Tabla 4. Verificación de la solución con las animaciones desactivadas.

3.6 Aprendizaje generado

El orden de dos líneas puede ser la diferencia entre que algo funcione y que no. La lección de fondo es más amplia: un error que solo aparece con una opción de accesibilidad activada es especialmente riesgoso, porque en el computador de desarrollo nunca se ve y, sin embargo, deja a la persona afectada sin poder usar la aplicación. De ahí se desprende una práctica que se incorporó al resto del proyecto: registrar siempre el escuchador antes de disparar la acción que emite el evento, y probar la interfaz con las animaciones desactivadas.

Sobre la mantenibilidad, la solución conserva la separación entre componentes: detalleDocumento.js sigue sin saber qué es un formulario; solo avisa que el usuario quiere editar un identificador determinado, y app.js decide qué hacer con ese aviso.

4. Accesibilidad y usabilidad

Indicador de logro: Formula una solución que incluye los conceptos de accesibilidad y usabilidad.

4.1 Criterios WCAG 2.1 aplicados

Criterio	Cómo se aplicó en la solución
1.1.1 Contenido no textual	Los íconos decorativos llevan aria-hidden; ninguna información depende solo de un ícono
1.3.1 Información y relaciones	HTML semántico, tabla con th scope y caption, fieldset con legend en el formulario
1.4.1 Uso del color	El estado de la conexión no se comunica solo con el punto de color: siempre hay texto
1.4.3 Contraste mínimo	Todos los textos superan la relación 4.5:1, medida y corregida (sección 4.2)
2.1.1 Teclado	Toda la aplicación se opera con teclado; ninguna acción depende del ratón
2.3.3 Animación	Se respeta prefers-reduced-motion para quien tenga desactivadas las animaciones
2.4.1 Evitar bloques	Enlace «Saltar al contenido principal» como primer elemento tabulable
2.4.3 Orden del foco	El orden de tabulación sigue el orden visual; al abrir una ventana el foco entra al primer campo
2.4.7 Foco visible	Contorno amarillo de 3 px en todo elemento enfocable; nunca se anula sin reemplazo
3.1.1 Idioma	Declarado en el elemento raíz del documento
3.3.1 a 3.3.3 Errores	Cada campo tiene su etiqueta; los errores describen qué corregir y aparecen junto al campo y en un resumen que recibe el foco
4.1.3 Mensajes de estado	Las zonas de carga, vacío y error están dentro de una región aria-live

Tabla 5. Criterios de accesibilidad aplicados.

4.2 Prueba de accesibilidad realizada

Se revisó la aplicación con un guion ejecutado sobre la interfaz en funcionamiento, que calcula la relación de contraste de cada texto según la fórmula de luminancia relativa de WCAG 2.1 y verifica etiquetas, encabezados, nombres accesibles y regiones dinámicas. La revisión detectó dos incumplimientos reales, ambos corregidos.

Criterio evaluado	Problema identificado	Mejora implementada	Resultado
1.4.3 Contraste mínimo	El texto descriptivo usaba la clase text-secondary de Bootstrap, que sobre el fondo de la aplicación alcanza 4.37:1, bajo el mínimo de 4.5:1	Se redefinió el color a un gris más oscuro, sin alterar el aspecto general	5.54:1, cumple
1.4.1 Uso del color	Todos los distintivos de tipo documental se veían del mismo gris: la regla general de CSS tenía más	Los colores se movieron a reglas de igual especificidad	Cada tipo se distingue, con contrastes entre 7.05:1 y 8.91:1

Criterio evaluado	Problema identificado	Mejora implementada	Resultado
	especificidad que las reglas de cada tipo y pisaba sus colores		

Tabla 6. Prueba de accesibilidad: criterio evaluado, problema y mejora.

El resto de la revisión no arrojó incumplimientos: ningún control de formulario sin etiqueta, ningún botón sin nombre accesible, un solo encabezado de primer nivel, sin saltos en la jerarquía y con el idioma correctamente declarado.

4.3 Prueba de navegación por teclado

Se recorrió el orden de tabulación completo. El primer elemento es el enlace para saltar al contenido, seguido de la acción principal, las tarjetas de resumen, los campos de filtro y las acciones de cada fila. El orden coincide con el orden visual y el foco es visible en todos los elementos.

4.4 Revisión de usabilidad

Durante el desarrollo se revisó la interfaz emulando una pantalla de celular de 375 por 812 píxeles y midiendo el tamaño real de los elementos. De ahí salieron dos mejoras, ya implementadas:

Oportunidad detectada	Cómo se detectó	Mejora implementada
En celular, el botón «Nuevo documento» quedaba dentro del menú desplegable, de modo que la acción principal exigía abrir el menú antes de poder usarla	Revisión de la interfaz en pantalla de 375 px	El botón salió del menú y está siempre visible en la barra superior; en pantallas angostas se abrevia a «+ Nuevo»
En pantallas bajas el formulario medía 737 px de alto contra 720 px de área visible, dejando los botones Guardar y Cancelar fuera de la vista	Medición del alto real del elemento contra el alto de la ventana	Se aplicó desplazamiento interno a la ventana, de modo que los botones queden siempre alcanzables

Tabla 7. Oportunidades de usabilidad detectadas y mejoras implementadas.

[PENDIENTE OBLIGATORIO: la evaluación exige además una revisión con al menos un usuario de prueba o persona externa al desarrollo. Las dos mejoras anteriores provienen de la revisión técnica propia, no de una prueba con un usuario real. Realice la prueba siguiendo el guion del README (sección 8) y registre aquí: participante, tarea asignada, dificultad observada y mejora implementada a partir de ese hallazgo.]

5. Integración y despliegue de los componentes

Indicador de logro: Gestiona el despliegue y la integración de los componentes en entornos complejos y multidisciplinarios.

Los nueve componentes se integraron hasta obtener una aplicación funcional, verificada contra el Backend real en ejecución. La integración es multidisciplinaria porque cruza dos asignaturas:

el Frontend de esta evaluación consume la API construida en Desarrollo Backend, que corre en otro proceso y en otro puerto.

5.1 Verificación del funcionamiento conjunto

Con ambos servidores levantados se ejecutó el ciclo completo, comprobando en el panel de red del navegador que cada acción disparara la petición correcta y que la respuesta se reflejara en pantalla:

Acción en la interfaz	Petición observada	Respuesta	Efecto en pantalla
Carga inicial	GET /api/salud	200	El indicador pasa a «API conectada»
Carga inicial	GET /api/tipos-documento	200	Se llenan los dos selectores de tipo
Carga inicial	GET /api/documentos	200	Se dibuja la tabla con los documentos
Carga inicial	GET /api/estadisticas	200	Se dibujan las tarjetas de resumen
Crear un documento	POST /api/documentos	201	Aviso de éxito, el total sube y la fila aparece
Buscar por texto	GET /api/documentos?buscar=...	200	El contador refleja las coincidencias
Pulsar Ver	GET /api/documentos/{id}	200	Se abre el detalle con todos los campos
Guardar cambios	PUT /api/documentos/{id}	200	Aviso de éxito y dato actualizado en la tabla
Confirmar eliminación	DELETE /api/documentos/{id}	200	Aviso de éxito, el total baja y la fila desaparece

Tabla 8. Verificación del funcionamiento conjunto de los componentes con el Backend.

5.2 Preflight CORS

Frontend y Backend corren en puertos distintos, por lo que el navegador envía una petición OPTIONS previa en cada llamada con cabecera Content-Type. Se verificó que el Backend responde 200 a esas peticiones, permitiendo que la llamada real se ejecute. Este es precisamente el tipo de detalle que solo aparece al integrar dos capas desarrolladas por separado.

5.3 Variables de configuración

Variable	Valor por defecto	Para qué sirve
URL_API	http://127.0.0.1:8010/api	Dirección del Backend; es lo único que hay que cambiar para apuntar a otro servidor
DOCUMENTOS_POR_PAGINA	10	Cuántos documentos trae cada página

Variable	Valor por defecto	Para qué sirve
ESPERA_BUSQUEDA	400	Milisegundos de pausa antes de buscar mientras se escribe
EXTENSIONES_PERMITIDAS	pdf, doc, docx, xls, xlsx, odt, jpg, jpeg, png	Extensiones aceptadas; debe coincidir con el Backend

Tabla 9. Variables de configuración, todas centralizadas en js/config.js.

5.4 Instalación, ejecución y despliegue

La aplicación son archivos estáticos: no requiere instalación de dependencias ni proceso de compilación. Se despliega copiando la carpeta a cualquier servidor web —Apache, Nginx, GitHub Pages o Netlify— y ajustando la variable URL_API con la dirección pública del Backend. El procedimiento completo y reproducible está documentado en el README del repositorio.

[OPCIONAL: si se realiza el despliegue en un entorno de prueba, agregar aquí la dirección pública y una captura de la aplicación funcionando en ese entorno.]

6. Buenas prácticas aplicadas al proyecto

Indicador de logro: *Elabora un detalle de buenas prácticas.*

Se documentan once buenas prácticas aplicadas efectivamente en el proyecto. Para cada una se indica su propósito, la forma en que se aplicó, un elemento observable o medible y su ubicación.

Buena práctica	Propósito y forma de aplicación	Elemento medible	Ubicación
1. Nomenclatura descriptiva	Que el código se entienda sin ejecutarlo. Las funciones que son acciones empiezan por verbo y los nombres coinciden con los del Backend	renderizarLista, abrirParaEditar, escaparHtml	Todo el proyecto
2. Separación de responsabilidades	Poder cambiar una parte sin romper las demás. Solo la capa de API llama a fetch; los componentes dibujan y avisan	0 llamadas a fetch fuera de documentosApi.js	js/api/ y js/componentes/
3. Estructura de carpetas por tipo	Encontrar rápido lo que se busca	css, js/api, js/componentes, vendor, docs	Raíz del proyecto
4. Reutilización de componentes	No escribir dos veces lo mismo. El mismo formulario crea y edita; los avisos se centralizan	1 formulario para 2 operaciones	formularioDocumento.js
5. Estado en un solo lugar	Evitar que dos partes tengan datos distintos. Un único objeto de estado, sin variables globales sueltas	0 variables globales	js/app.js
6. Validación antes de enviar	Avisar de inmediato y no gastar una petición en datos que serán	5 reglas validadas antes del envío	función validar()

Buena práctica	Propósito y forma de aplicación	Elemento medible	Ubicación
	rechazados. Mismas reglas que el Backend		
7. Errores sin detalle técnico	Que el usuario entienda qué pasó. Nunca se muestran trazas ni rutas internas	0 trazas expuestas al usuario	documentosApi.js, mensajes.js
8. Comentarios que explican el porqué	Que quien retome el proyecto entienda las decisiones, no la sintaxis	Cada decisión no evidente está justificada	Todos los archivos de js/
9. Configuración centralizada	Cambiar de servidor sin buscar valores dispersos	Mover la aplicación exige cambiar 1 línea	js/config.js
10. HTML semántico	Que la estructura sea comprensible para lectores de pantalla sin parches ARIA	1 solo h1, 0 saltos de nivel, 13 celdas con scope, 0 controles sin etiqueta	index.html
11. Dependencias versionadas localmente	Que la aplicación funcione igual sin internet y sin depender de un servicio externo	0 peticiones a dominios externos al cargar	vendor/bootstrap/

Tabla 10. Buenas prácticas aplicadas, con su elemento observable y su ubicación.

7. Control de versiones

Indicador de logro: *Gestiona la colaboración y trabajo de equipo con herramientas de control de versiones.*

El proyecto se desarrolló de forma individual, se versionó con Git y se publicó en un repositorio público de GitHub:

<https://github.com/Luisorellanacabrera/gestor-documental-frontend>

Aspecto	Cómo se gestionó
Mensajes de commit	Descriptivos: explican qué cambió y por qué, nunca «cambios» o «avance»
Alcance de cada commit	Acotado a un cambio con sentido propio, dejando la aplicación en un estado que funciona
Orden del historial	Refleja cómo se construyó la aplicación: configuración, estructura, capa de datos, componentes e integración
Uso de ramas	Historial lineal sobre main, por la razón que se explica más abajo

Tabla 11. Gestión del control de versiones.

La evaluación pide utilizar ramas «cuando sean pertinentes». En un desarrollo individual y secuencial, sin trabajo paralelo ni integraciones que resolver, una rama por funcionalidad habría sido una formalidad sin función: no hay conflictos que prevenir ni revisiones de otra persona que esperar. Se optó por un historial lineal con commits acotados, que resulta más legible para quien

deba retomar el proyecto. Al tratarse de un trabajo individual, la evidencia de este indicador es la trazabilidad del historial y no la coordinación entre integrantes.

[*COMPLETAR: agregar una captura del historial de commits del repositorio.*]

8. Diseño de la interfaz de usuario

Indicador de logro: Diseña interfaces de usuario interactivas, comprensibles e intuitivas, incorporando conceptos de diseño centrado en el usuario y usabilidad.

La interfaz se diseñó pensando en quién la va a usar: el equipo directivo de una escuela rural, sin perfil técnico, que hoy administra sus documentos en papel. De ahí que el vocabulario de la aplicación sea el de la escuela —memos, oficios, citas— y no el del sistema.

8.1 Operaciones disponibles

Operación	Cómo se realiza
Visualizar	Tabla con todos los documentos, paginada de a diez
Consultar el detalle	Botón «Ver» que abre una ventana con toda la información del documento
Crear	Botón «Nuevo documento», siempre visible en la barra superior
Editar	Botón «Editar» en la fila, o desde la ventana de detalle
Eliminar	Botón «Eliminar» con confirmación previa que muestra el título del documento
Buscar	Campo de búsqueda por título y descripción, que se aplica solo al dejar de escribir
Filtrar por tipo	Selector de tipo documental, más las tarjetas de resumen que filtran al pulsarlas
Filtrar por fecha	Rango desde y hasta

Tabla 12. Operaciones que la interfaz permite realizar.

8.2 Información del listado

El listado muestra las cinco columnas exigidas: título del documento, tipo, fecha, descripción o referencia y las acciones disponibles. Cuando un documento tiene archivo asociado, junto al título aparece un distintivo con su nombre.

8.3 Retroalimentación visual

Situación	Qué ve el usuario
Cargando	Indicador giratorio con el texto «Cargando documentos...» y la tabla oculta
Operación exitosa	Aviso emergente verde: documento creado, actualizado o eliminado
Guardando	El botón se bloquea, muestra un indicador y cambia a «Guardando...», evitando el doble envío

Situación	Qué ve el usuario
Datos inválidos	Resumen de errores al inicio del formulario, que recibe el foco, más el mensaje bajo cada campo
Sin resultados	Mensaje distinto según haya filtros puestos o el listado esté realmente vacío
Servidor caído	Mensaje en lenguaje común con un botón para reintentar
Estado de la conexión	Indicador permanente en la barra superior, con punto de color y texto

Tabla 13. Retroalimentación visual entregada al usuario en cada situación.

8.4 Diseño adaptable

La interfaz se adapta a distintos tamaños de pantalla. En celular las tarjetas de resumen se reorganizan en dos columnas, la columna de descripción se oculta para que la tabla siga siendo legible —el dato permanece disponible en el detalle— y la acción principal se abrevia sin perder visibilidad. Se verificó en pantallas de 375, 768 y 1280 píxeles de ancho.

[*COMPLETAR: insertar capturas de la aplicación en pantalla de escritorio y de celular, numeradas según APA, y la evidencia de la revisión con el usuario de prueba.*]

9. Estrategias de optimización

Indicador de logro: *Aplica estrategias de optimización avanzadas.*

Se implementaron y documentaron cuatro optimizaciones. Todas apuntan al mismo objetivo práctico: que la aplicación funcione con soltura en el notebook disponible en la escuela.

9.1 Búsqueda con pausa

Aspecto	Detalle
Situación inicial	El listado se recargaba en cada pulsación de tecla. Escribir «apoderados», de diez letras, generaba diez peticiones al servidor, de las cuales nueve quedaban obsoletas de inmediato
Optimización aplicada	La función <code>retrasar()</code> espera 400 milisegundos sin escritura antes de consultar
Beneficio medido	Escribir «apoderados» genera una sola petición: una reducción del 90 % en ese caso, verificada en el panel de red del navegador. Además la lista deja de parpadear mientras se escribe

Tabla 14. Optimización 1: búsqueda con pausa.

9.2 Catálogo de tipos en memoria

Aspecto	Detalle
Situación inicial	El catálogo de tipos se necesita en el filtro y en el formulario. Pedirlo en cada apertura significaba una petición por vez, para datos que cambian una vez al año
Optimización aplicada	La función que obtiene los tipos guarda el resultado la primera vez; las llamadas siguientes lo devuelven sin tocar la red

Aspecto	Detalle
Beneficio medido	El catálogo se solicita una sola vez por sesión, verificado en el panel de red. Abrir el formulario es inmediato

Tabla 15. Optimización 2: catálogo en memoria.

9.3 Delegación de eventos

Aspecto	Detalle
Situación inicial	Con tres botones por fila y diez filas, registrar un evento por botón implicaba treinta escuchadores, que además había que recrear en cada búsqueda porque las filas se redibujan
Optimización aplicada	Un único escuchador en el cuerpo de la tabla identifica el botón pulsado. El mismo patrón se usa en las tarjetas de resumen
Beneficio	De treinta escuchadores a uno, y no se recrean al redibujar: menos memoria y un redibujado más liviano

Tabla 16. Optimización 3: delegación de eventos.

9.4 Un solo redibujado de la tabla

Aspecto	Detalle
Situación inicial	Insertar las filas una por una obliga al navegador a recalcular el diseño de la página en cada vuelta del ciclo
Optimización aplicada	Se arma todo el HTML en una cadena y se asigna una única vez
Beneficio	Un recálculo de diseño en lugar de diez

Tabla 17. Optimización 4: un solo redibujado.

10. Integración del framework en la interfaz

Indicador de logro: Integra el uso de frameworks en la interfaz de usuario.

La mayor parte de la interfaz y de sus funcionalidades está construida con los mecanismos propios del framework definido en la asignatura, manteniendo consistencia técnica y estructural durante todo el proyecto.

Elemento del framework	Uso en la aplicación
Sistema de rejilla	Tarjetas de resumen y campos de la barra de filtros, con adaptación por tamaño de pantalla
Componente de tabla	Listado de documentos con encabezado, filas destacables y desplazamiento horizontal
Ventanas modales	Detalle, formulario de crear y editar, y confirmación de eliminación
Avisos emergentes	Retroalimentación de las operaciones exitosas y de los errores
Componentes de formulario	Etiquetas, campos, texto de ayuda y estados de validación
Clases de utilidad	Espaciado, alineación y visibilidad condicional por tamaño de pantalla

Elemento del framework	Uso en la aplicación
Módulos de JavaScript	Cada componente es un módulo con importaciones y exportaciones explícitas
Eventos del framework	Se usan los eventos del ciclo de vida de las ventanas modales para encadenarlas y para gestionar el foco

Tabla 18. Mecanismos del framework utilizados en la interfaz.

La consistencia estructural se mantiene en todo el proyecto: todos los componentes siguen el mismo patrón —un módulo que exporta funciones de dibujo y funciones de conexión de eventos— y ninguno introduce una forma distinta de resolver lo que el framework ya resuelve. Los estilos propios se limitan a la identidad visual del establecimiento y a los ajustes de accesibilidad, sin reescribir lo que Bootstrap ya entrega.

11. Consumo de endpoints

Indicador de logro: Aplica el consumo de endpoints.

Todo el consumo pasa por un único módulo, `documentosApi.js`, de modo que si cambia una URL o el formato de la respuesta se corrige en un solo lugar. Se consumen los seis endpoints exigidos más dos de apoyo.

Método y URL	Propósito	Información enviada	Información recibida
GET /documentos	Listar con filtros y paginación	buscar, tipo, desde, hasta, pagina y limite en la URL	Arreglo de documentos y datos de paginación
GET /documentos/{id}	Obtener un documento para verlo o editarlo	Identificador en la ruta	El documento completo
POST /documentos	Crear un documento	titulo, tipo, fecha_documento, descripcion y archivo_nombre en el cuerpo	Documento creado, con su identificador
PUT /documentos/{id}	Actualizar un documento	Identificador y los mismos campos que POST	Documento actualizado
DELETE /documentos/{id}	Eliminar un documento	Identificador en la ruta	Confirmación e identificador eliminado
GET /tipos-documento	Llenar los selectores de tipo	—	Identificador, código y nombre de cada tipo
GET /estadisticas	Alimentar las tarjetas de resumen	—	Total de documentos y cantidad por tipo

Método y URL	Propósito	Información enviada	Información recibida
GET /salud	Comprobar que el Backend responde antes de pedir datos	—	Estado del servicio

Tabla 19. Endpoints consumidos, con la información enviada y recibida.

11.1 Manejo de solicitudes, estados y errores

Situación	Cómo se maneja
Estado de carga	Indicador giratorio mientras la petición está en curso, con la tabla oculta
Ausencia de resultados	Mensaje distinto según haya filtros aplicados o el listado esté vacío
Error de validación (400 y 422)	Los errores del servidor se muestran dentro del formulario, campo por campo, más un resumen
Recurso inexistente (404)	Aviso emergente con el mensaje que entrega el servidor
Servidor caído o sin red	La excepción de fetch se traduce a un mensaje en lenguaje común, con botón de reintentar
Respuesta sin contenido (204)	Se maneja de forma explícita, sin intentar interpretar un cuerpo vacío
Respuesta con formato inesperado	Se captura el fallo al interpretar el JSON y se informa sin exponer detalles técnicos

Tabla 20. Manejo de los estados y errores de la comunicación con la API.

Para transportar la información de los fallos se definió una clase de error propia que conserva el código HTTP y el arreglo de errores de validación del Backend. Gracias a eso, la aplicación puede distinguir un error de validación —que debe mostrarse dentro del formulario, junto al campo correspondiente— de un error general, que se muestra como aviso emergente.

12. Estrategias de seguridad

Indicador de logro: Implementa estrategias de seguridad avanzada.

Medida implementada	Riesgo que reduce	Ubicación
Escape de HTML en todo dato mostrado	Cross-site scripting almacenado: si alguien guardara como título una etiqueta con código, al pintarla el navegador la ejecutaría. Al convertir los caracteres especiales en entidades, el texto se muestra literal	utilidades.js
Validación de los datos ingresados	Envío de datos mal formados y errores evitables: se valida largo, formato de fecha y extensión antes de enviar	formularioDocumento.js
Validación de la extensión del archivo	Registro de archivos ejecutables: solo se aceptan las extensiones ofimáticas y de imagen definidas	config.js
Mensajes de error sin detalle técnico	Divulgación de información útil para un atacante: nunca se muestran trazas, rutas internas ni consultas	documentosApi.js

Medida implementada	Riesgo que reduce	Ubicación
Sin credenciales en el código fuente	Filtración de secretos: todo lo que está en el Frontend viaja al navegador y cualquiera puede leerlo, por lo que allí no se guarda nada sensible	config.js
Framework servido localmente	Dependencia de terceros y ataques a la cadena de suministro: un CDN comprometido o caído no afecta a la escuela	vendor/
Confirmación en acciones destructivas	Borrado accidental: eliminar exige confirmación mostrando el título, y el foco parte en «Cancelar»	confirmarEliminar.js
Bloqueo del botón mientras se guarda	Registros duplicados por doble clic o por una conexión lenta	formularioDocumento.js

Tabla 21. Medidas de seguridad implementadas y riesgo que reduce cada una.

12.1 Alcance actual y limitación declarada

La aplicación todavía no cuenta con autenticación: quien conozca la dirección puede ver y modificar los documentos. La escuela indicó que aún no tiene protocolos de seguridad definidos, por tratarse de una idea nueva. Se declara explícitamente porque es una limitación real y no un descuido, y porque un Frontend no puede resolverla por sí solo: requiere que el Backend implemente autenticación y control de acceso. Queda registrada como la primera prioridad de la retrospectiva.

13. Documentación y difusión del conocimiento

Indicador de logro: Concluye en una documentación y difusión del conocimiento.

La documentación del proyecto se organizó en el repositorio, de modo que quien lo retome encuentre todo junto al código y no en un archivo aparte que puede quedar desactualizado.

Documento	Contenido
README.md	Descripción del problema y la solución, tecnologías, estructura, componentes, instalación, ejecución, endpoints, buenas prácticas, seguridad, optimizaciones, problema técnico resuelto, dificultades y retrospectiva
docs/evidencia_integracion.md	Ciclo completo verificado contra el Backend, con la petición observada y el efecto en pantalla de cada acción
docs/evidencia_accesibilidad.md	Método, mediciones de contraste antes y después, y prueba de navegación por teclado
docs/evidencia_optimizacion.md	Situación inicial, optimización aplicada y beneficio medido de cada una
Comentarios en el código	Explican el porqué de las decisiones no evidentes, no la sintaxis
Este informe	Documento formal con los trece indicadores de la unidad

Tabla 22. Documentación generada durante el proyecto.

[PENDIENTE OBLIGATORIO: la evaluación exige una breve demostración de la solución, explicando el funcionamiento general, los componentes principales, la integración con el Backend, las decisiones técnicas relevantes y los aprendizajes. Grabe el video o prepare la presentación, y registre aquí el enlace.]

14. Retrospectiva y mejora continua

Indicador de logro: Establece un liderazgo en la mejora continua.

14.1 Cómo se tomó la decisión

El proyecto se desarrolló de forma individual, por lo que la priorización no requirió acuerdo entre integrantes, sino un criterio explícito y aplicable de forma consistente. El criterio utilizado fue cruzar el impacto para el equipo directivo de la escuela con el esfuerzo de implementación, sobre las oportunidades detectadas durante el desarrollo y las pruebas.

Con ese criterio se ordenaron las cinco oportunidades de la tabla siguiente y se implementó de inmediato la de mayor impacto y menor esfuerzo. Las de prioridad alta que quedaron pendientes —autenticación y carga de archivos— no dependen solo del Frontend: requieren que el Backend las exponga primero, y por eso se registran como trabajo de la siguiente iteración y no como omisiones de esta entrega.

14.2 Oportunidades de mejora identificadas

#	Situación o problema	Mejora propuesta	Prioridad	Responsable	Acción a realizar
1	Cualquiera que abra la dirección puede ver y modificar documentos con datos de apoderados y del personal	Autenticación e inicio de sesión con control de acceso	Alta	Luis Orellana	Coordinar con la capa Backend la creación de usuarios y proteger las rutas del Frontend
2	Solo se registra el nombre del archivo; el documento físico sigue fuera del sistema	Carga real del archivo y descarga desde el detalle	Alta	Luis Orellana	Agregar un campo de tipo archivo y consumir el endpoint de carga cuando el Backend lo exponga
3	En celular la tabla oculta la columna de descripción y las acciones quedan apretadas	Vista en tarjetas para pantallas pequeñas	Media	Luis Orellana	Crear una variante del componente de listado que dibuje tarjetas bajo los 768 píxeles
4	Si se elimina un documento por error no hay forma de recuperarlo	Papelera con eliminación lógica y opción de restaurar	Media	Luis Orellana	Requiere un campo de estado en el Backend; en el Frontend, una vista de papelera

#	Situación o problema	Mejora propuesta	Prioridad	Responsable	Acción a realizar
5	El botón principal estaba escondido en el menú desplegable en celulares	Dejarlo siempre visible en la barra superior	Alta	Luis Orellana	Implementada en esta entrega

Tabla 23. Oportunidades de mejora para la siguiente iteración.

14.3 Mejora implementada dentro del alcance

Se implementó la mejora número 5, surgida de la revisión de usabilidad: era la de mayor impacto inmediato para el usuario real y la de menor esfuerzo técnico. El botón «Nuevo documento» salió del menú desplegable y quedó siempre visible, abreviado en pantallas angostas.

También se implementaron, dentro del mismo alcance, las dos correcciones de accesibilidad de la sección 4 y el ajuste de desplazamiento interno del formulario, todas surgidas de las pruebas realizadas sobre la aplicación en funcionamiento.

15. Conclusiones

Se construyó la capa Frontend del Gestor Documental de la Escuela Básica G-733 Chorombo Bajo, cubriendo los trece indicadores de la unidad: se propuso y documentó una estructura de nueve componentes con responsabilidades separadas; se resolvió y documentó un problema técnico real de integración entre ventanas modales; se aplicaron criterios de accesibilidad WCAG 2.1 verificados con mediciones; se integraron los componentes contra el Backend real; se documentaron once buenas prácticas; se versionó el proyecto en un repositorio público; se diseñó una interfaz que permite las siete operaciones exigidas con retroalimentación visual clara; se implementaron cuatro optimizaciones, dos de ellas con medición; se construyó la interfaz con los mecanismos del framework; se consumieron ocho endpoints con manejo completo de estados y errores; se aplicaron ocho medidas de seguridad; y se registró una retrospectiva con cinco oportunidades de mejora.

El aprendizaje más valioso no fue técnico sino de criterio: las restricciones del cliente son también restricciones de diseño. Que la escuela cuente con un solo notebook básico y sin infraestructura previa determinó que la aplicación no necesite instalación, no dependa de un gestor de paquetes y funcione sin conexión a internet. Una solución técnicamente más ambiciosa, pero imposible de instalar y mantener en el establecimiento, no habría resuelto el problema real.

En lo técnico, el hallazgo más significativo fue comprobar que un defecto puede manifestarse solo cuando el usuario tiene activada una opción de accesibilidad. Eso obligó a incorporar la prueba con animaciones desactivadas como parte de la verificación, y dejó una lección aplicable

a cualquier proyecto: la accesibilidad no es una capa que se agrega al final, sino una condición que también revela errores de funcionamiento.

Respecto de la integración con la asignatura Desarrollo Backend, la aplicación consume el contrato definido en esa capa y maneja sus respuestas, sus códigos de estado y sus errores de validación. Ambas evaluaciones quedan así integradas como dos capas del mismo proyecto, tal como solicita el enunciado.

16. Declaración de uso de Inteligencia Artificial

DECLARACIÓN EXPRESA: en la elaboración de este proyecto y de este informe SÍ se utilizaron herramientas de inteligencia artificial generativa. A continuación se identifica la herramienta empleada, las tareas en que se utilizó y la justificación técnica de esa decisión.

Herramienta empleada: Claude, modelo de lenguaje desarrollado por Anthropic, utilizado a través de la interfaz de línea de comandos Claude Code, que permite al asistente leer y escribir archivos del proyecto y ejecutar comandos en el entorno de desarrollo local.

16.1 Tareas en que se utilizó

Tarea	Nivel de uso
Análisis del caso y definición de los requerimientos a partir de la ficha del actor de interés	Trabajo propio; la IA no participó.
Propuesta de la estructura de componentes y sus responsabilidades	Propuesta por la IA y revisada, ajustada y validada por el estudiante.
Escritura del código de la interfaz, los componentes y la capa de consumo de la API	Generado con apoyo de la IA sobre la estructura definida en la asignatura.
Pruebas de accesibilidad, contraste y navegación por teclado	Guiones generados con apoyo de la IA; los resultados fueron revisados y las correcciones verificadas por el estudiante.
Redacción del README y de este informe	Redactado con apoyo de la IA y revisado por el estudiante.
Verificación funcional del ciclo CRUD contra el Backend en ejecución	Ejecutada y comprobada por el estudiante.

Tabla 24. Tareas del proyecto y grado de participación de la herramienta de IA.

16.2 Justificación técnica del uso

Se optó por esta herramienta por tres razones de orden técnico. Primero, permitió acelerar tareas repetitivas y de baja densidad conceptual —la documentación, las tablas de endpoints y los guiones de verificación—, liberando tiempo para el análisis del caso y la comprobación funcional, que es donde reside el aprendizaje de la unidad.

Segundo, se utilizó como instancia de contraste de las decisiones de diseño: la separación de responsabilidades entre componentes, la decisión de encadenar ventanas modales mediante eventos en lugar de temporizadores y la elección de escapar el HTML en un único punto fueron discutidas con el asistente antes de implementarse, lo que obligó a explicitar los criterios que sustentan cada decisión y a dejarlos documentados.

Tercero, se empleó como apoyo en la depuración. Los defectos corregidos —el contraste insuficiente, la especificidad de CSS que anulaba los colores por tipo, la etiqueta del botón que no se restauraba y el fallo de las ventanas encadenadas con las animaciones desactivadas— fueron detectados ejecutando y midiendo la aplicación, no aceptando el código como correcto. En todos los casos la corrección se verificó volviendo a medir sobre la aplicación en funcionamiento, criterio considerado indispensable: el código generado por una IA no se da por correcto hasta que la comprobación lo confirma.

16.3 Alcance y responsabilidad

El estudiante declara que comprende el funcionamiento de la totalidad del código entregado y asume la responsabilidad sobre su contenido. La herramienta se utilizó como instrumento de apoyo y no como sustituto del trabajo propio: la definición del problema, la validación de que la solución responde a la necesidad planteada por la Escuela Básica G-733 Chorombo Bajo y la comprobación funcional de la aplicación fueron realizadas y verificadas personalmente.

Nota: *si el uso real de la herramienta fue distinto al descrito, ajuste la tabla 24 y la sección 16.2 antes de entregar, y borre esta nota.*

17. Referencias

Anthropic. (2025). Claude [modelo de lenguaje de gran escala].

<https://www.anthropic.com/claude>

Bootstrap. (2021). Bootstrap 5.1 documentation. <https://getbootstrap.com/docs/5.1/>

Escuela Básica G-733 Chorombo Bajo. (2025). Ficha VcM actor externo: Información actor de interés externo. Comuna de María Pinto.

Mozilla. (2025). MDN Web Docs: JavaScript modules.

<https://developer.mozilla.org/es/docs/Web/JavaScript/Guide/Modules>

World Wide Web Consortium. (2018). Web Content Accessibility Guidelines (WCAG) 2.1.

<https://www.w3.org/TR/WCAG21/>